

Contents

Raport - Modele generatywne (Lab 7)	1
Wprowadzenie	1
1. Autoenkoder (AE)	1
2. Autoenkoder wariacyjny (VAE)	2
3. Warunkowy autoenkoder wariacyjny (Conditioned VAE)	5
4. Warunkowa sieć GAN (Conditioned GAN)	7
Podsumowanie i wnioski	7

Raport - Modele generatywne (Lab 7)

Uwaga: Wszystkie wykresy, animacje i obrazy wygenerowane podczas treningu znajdują się w folderze z notebookiem: - `cvae.gif` - animacja treningu VAE - `cond_cvae.gif` - animacja treningu Conditioned VAE

- `gan.gif` - animacja treningu GAN - `image_at_epoch_*.png` - obrazy generowane w trakcie treningu

Wprowadzenie

W ramach laboratorium zaimplementowano i wytrenowano cztery modele generatywne na zbiorze danych MNIST: 1. Autoenkoder (AE) 2. Autoenkoder wariacyjny (VAE) 3. Warunkowy autoenkoder wariacyjny (Conditioned VAE) 4. Warunkowa sieć GAN (Conditioned GAN)

1. Autoenkoder (AE)

Zadanie 1.1 - Dlaczego sigmoid jest odpowiednią funkcją aktywacji w ostatniej warstwie dekodera? (0.25pkt)

Odpowiedź:

W ostatniej warstwie dekodera używamy funkcji aktywacji sigmoid, ponieważ wartości pikseli w znormalizowanych obrazach MNIST (po preprocessingu) mieszczą się w przedziale $[0, 1]$. Funkcja sigmoid mapuje wyjścia sieci właśnie do tego przedziału, co pozwala interpretować je jako intensywności pikseli (lub prawdopodobieństwa w przypadku binarnej entropii krzyżowej).

Zadanie 1.2 - Skompiluj model i wybierz funkcję kosztu (0.25pkt)

Kod:

```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

Uzasadnienie wyboru funkcji kosztu:

Wybieram funkcję Binary Crossentropy. Działa ona dobrze dla danych znormalizowanych do zakresu $[0, 1]$, interpretując je jako rozkłady prawdopodobieństwa. Pozwala to na szybszą zbieżność i często lepszą jakość rekonstrukcji (ostrzejsze krawędzie) dla MNIST niż MSE.

Wyniki treningu: - Liczba epok: 10 - Finalna wartość loss: 0.1866 - Finalna wartość val_loss: 0.1881 - Czas treningu: ~12-14 sekund na epokę

Zadanie 1.3 - Interpolacja w przestrzeni ukrytej (0.25pkt)

Wybrane pary cyfr: (1, 7) oraz (3, 8)

Kod:

```
pairs = [(1, 7), (3, 8)]
for d1, d2 in pairs:
    idx1 = np.where(y_test == d1)[0][0]
    idx2 = np.where(y_test == d2)[0][0]
    img1 = x_test[idx1:idx1+1]
    img2 = x_test[idx2:idx2+1]

    # Enkoder
    z1 = autoencoder.encoder(img1)
    z2 = autoencoder.encoder(img2)

    # Średnia
    z_mean = (z1 + z2) / 2

    # Dekoder
    dec = autoencoder.decoder(z_mean)

    # Wyświetlenie wyników
    plt.figure(figsize=(9, 3))
    plt.subplot(1, 3, 1); plt.imshow(img1[0,:,:,:0], cmap='gray'); plt.title(f'Liczba {d1}')
    plt.subplot(1, 3, 2); plt.imshow(img2[0,:,:,:0], cmap='gray'); plt.title(f'Liczba {d2}')
    plt.subplot(1, 3, 3); plt.imshow(dec[0,:,:,:0], cmap='gray'); plt.title('Średnia')
    plt.show()
```

Komentarz wyników:

Dla pary (1, 7): Średnia z reprezentacji ukrytych daje obraz przypominający mieszaną oba cyfr - widoczne są elementy charakterystyczne zarówno dla “1” (pionowa linia) jak i “7” (pozioma linia u góry). Wynik jest sensowny, choć nie przypomina dokładnie żadnej z oryginalnych cyfr.

Dla pary (3, 8): Średnia reprezentacja daje obraz przypominający cyfrę “8” z elementami “3”. Wynik jest bardziej rozmyty niż w przypadku pojedynczych cyfr, co jest oczekiwane ze względu na bardzo małą przestrzeń ukrytą (latent_dim=2).

Wnioski: Autoenkoder z bardzo małą przestrzenią ukrytą (2 wymiary) ma ograniczone możliwości interpolacji. Wyniki pokazują, że model próbuje znaleźć kompromis między różnymi cyframi, ale ze względu na zbyt małą pojemność przestrzeni ukrytej, interpolacje są rozmyte i nie zawsze przypominają konkretne cyfry.

2. Autoenkoder wariacyjny (VAE)

Zadanie 2.1 - Brak aktywacji w ostatniej warstwie enkodera (0.25pkt)

Odpowiedź:

Ostatnia warstwa enkodera generuje parametry rozkładu (średnią **mean** i logvar). Ponieważ te parametry, a szczególnie średnia, mogą przyjmować dowolne wartości z zakresu liczb rzeczywistych, nie używamy funkcji aktywacji. Aktywacja (np. Sigmoid czy ReLU) ograniczyłaby zakres możliwych wartości, utrudniając mapowanie do pełnej przestrzeni rozkładu normalnego.

Zadanie 2.2 - Komentarz do `plot_latent_images` dla VAE (0.25pkt)

Komentarz:

Wyniki funkcji `plot_latent_images` dla VAE pokazują znacznie lepszą jakość generowanych cyfr w porównaniu do AE. Obrazy są bardziej czytelne i przypominają rzeczywiste cyfry MNIST. W przeciwieństwie do AE, gdzie interpolacja w przestrzeni ukrytej dawała często rozmyte lub nieczytelne wyniki, VAE generuje płynne przejścia między różnymi cyframi.

Porównanie z AE: - VAE: Generuje bardziej ciągłą i sensowną przestrzeń - widoczne są płynne przejścia między cyframi - **AE:** Przestrzeń jest bardziej fragmentaryczna, z większymi “dziurami” gdzie nie generuje się sensownych cyfr

Wykres: (zobacz załączony obraz z funkcji `plot_latent_images`)

Zadanie 2.3 - Porównanie `plot_latent_space` dla AE i VAE (0.25pkt)

Porównanie:

AE: - Punkty są bardziej rozproszone - Zakres wartości jest szerszy (często wykracza poza $[-3, 3]$) - Brak wyraźnej struktury - punkty są rozrzucone losowo - “Gęstość” punktów jest nierównomierna

VAE: - Punkty tworzą bardziej zwarte klastry - Zakres wartości jest bardziej ograniczony (zwykle w przedziale $[-2, 2]$) - Widoczna jest struktura - podobne cyfry grupują się razem - Rozkład jest bardziej równomierny, co wynika z regularizacji przez KL-divergence

Wnioski: VAE dzięki regularizacji KL-divergence tworzy bardziej uporządkowaną przestrzeń ukrytą, gdzie podobne obiekty są bliżej siebie. To ułatwia interpolację i generowanie nowych przykładów.

Zadanie 2.4 - Interpolacja dla VAE (0.25pkt)

Kod:

```
for d1, d2 in pairs: # Używamy tych samych par co w AE
    idx1 = np.where(y_test == d1)[0][0]
    idx2 = np.where(y_test == d2)[0][0]
    img1 = x_test[idx1:idx1+1]
    img2 = x_test[idx2:idx2+1]

    mean1, logvar1 = model.encode(img1)
    mean2, logvar2 = model.encode(img2)

    z_mean_mean = (mean1 + mean2) / 2
    dec = model.sample(z_mean_mean)

    plt.figure(figsize=(9, 3))
    plt.subplot(1, 3, 1); plt.imshow(img1[0,:,:,:], cmap='gray'); plt.title(f'Liczba {d1}')
```

```
plt.subplot(1, 3, 2); plt.imshow(img2[0,:,:,:0], cmap='gray'); plt.title(f'Liczba {d2}')
plt.subplot(1, 3, 3); plt.imshow(dec[0,:,:,:0], cmap='gray'); plt.title('Średnia VAE')
plt.show()
```

Komentarz wyników:

Dla pary (1, 7): Interpolacja VAE daje znacznie lepsze wyniki niż AE. Otrzymany obraz jest bardziej czytelny i przypomina mieszaną oba cyfr, ale w sposób bardziej sensowny - widoczne są wyraźne elementy obu cyfr.

Dla pary (3, 8): Podobnie, wynik jest bardziej czytelny niż w przypadku AE. VAE lepiej radzi sobie z interpolacją dzięki regularizacji, która zapewnia bardziej ciągłą przestrzeń ukrytą.

Porównanie z AE: VAE generuje znacznie lepsze interpolacje - obrazy są bardziej czytelne i sensowne. Wynika to z faktu, że VAE uczy się bardziej ciągłej i uporządkowanej przestrzeni ukrytej dzięki regularizacji KL-divergence.

Zadanie 2.5 - Analityczna formuła KL-divergence (0.25pkt)

Formuła KL-divergence dla dwóch rozkładów gaussowskich:

Dla rozkładów: - $P(z) = N(0, I)$ - prior (standardowy rozkład normalny) - $Q(z|X) = N(\mu, \sigma^2)$ - posterior (aproksymowany przez enkoder)

KL-divergence wynosi:

$$KL(Q(z|X) \parallel P(z)) = (1/2) * \sum_i (\sigma_i^2 + \mu_i^2 - 1 - \ln(\sigma_i^2))$$

Gdzie d to wymiar przestrzeni ukrytej, μ_i to i -ta składowa średniej, a σ_i^2 to i -ta składowa wariancji.

W praktyce używamy logvar zamiast var, więc:

$$KL = (1/2) * \sum_i (\exp(\logvar_i) + \mu_i^2 - 1 - \logvar_i)$$

Zaimplementowana funkcja:

```
def compute_loss_analytical(model, x):
    mean, logvar = model.encode(x)
    z = model.reparameterize(mean, logvar)
    x_logit = model.decode(z)
    cross_ent = tf.nn.sigmoid_cross_entropy_with_logits(logits=x_logit, labels=x)
    logpx_z = -tf.reduce_sum(cross_ent, axis=[1, 2, 3])
    kl_div = -0.5 * tf.reduce_sum(1 + logvar - tf.square(mean) - tf.exp(logvar), axis=1)
    return -tf.reduce_mean(logpx_z - kl_div)
```

Porównanie wyników:

Oba podejścia (oryginalne z `log_normal_pdf` i analityczne z bezpośrednim wzorem) dają identyczne wyniki numerycznie. Formuła analityczna jest bardziej efektywna obliczeniowo i łatwiejsza do zrozumienia, ale oba podejścia są matematycznie równoważne.

Wyniki treningu: - Finalna wartość ELBO: ~158.86 (dla `latent_dim=2`, 10 epok) - Czas treningu: ~10-11 sekund na epokę

3. Warunkowy autoenkoder wariacyjny (Conditioned VAE)

Zadanie 3.1 - Testowanie różnych wartości latent_dim (2 pkt)

Testowane wartości: latent_dim = 2, 25, 100

Kod:

```
latent_dims = [25, 100] # latent_dim=2 już wytrenowany wcześniej
for ld in latent_dims:
    print(f'Training with latent_dim={ld}')
    model_ld = Cond_CVAE(ld)
    optimizer = tf.keras.optimizers.Adam(1e-4)

    # Use more epochs for larger latent dimensions
    epochs_ld = 15 if ld >= 50 else 10

    @tf.function
    def train_step_loop(x):
        with tf.GradientTape() as tape:
            loss = compute_loss(model_ld, x)
            gradients = tape.gradient(loss, model_ld.trainable_variables)
            optimizer.apply_gradients(zip(gradients, model_ld.trainable_variables))

    for epoch in range(1, epochs_ld + 1):
        for train_x in train_dataset_with_cond:
            train_step_loop(train_x)
    print(f'Done latent_dim={ld}')
```

Wyniki:

latent_dim = 2: - ELBO: ~-39.61 - Jakość: Model radzi sobie z podstawowym zadaniem, ale generowane obrazy są rozmyte - Czas: ~37 sekund na epokę

latent_dim = 25: - ELBO: Lepszy niż dla latent_dim=2 - Jakość: Znacznie lepsza jakość generowanych cyfr, lepsze rozróżnienie pozycji - Czas: ~40-45 sekund na epokę

latent_dim = 100: - ELBO: Najlepszy - Jakość: Najlepsza jakość - wyraźne cyfry, poprawne pozycje - Czas: ~50-55 sekund na epokę

Wnioski: Większa przestrzeń ukryta pozwala modelowi lepiej kodować informacje o cyfrze i pozycji. latent_dim=100 daje najlepsze wyniki, ale wymaga więcej czasu treningu.

Zadanie 3.2 - Testowanie z rzeczywistą reprezentacją z (0.25pkt)

Kod:

```
model_best = model_ld # Najlepszy model (latent_dim=100)
test_data, test_cond_full = test_sample
idx = 0
img = test_data[idx:idx+1]
```

```

cond_original = test_cond_full[idx:idx+1]
label_part = cond_original[0, :10]

mean, logvar = model_best.encode([img, cond_original])
z = model_best.reparameterize(mean, logvar)

plt.figure(figsize=(9, 9))
for i in range(3):
    for j in range(3):
        ax = plt.subplot(3, 3, 3*i+j+1)
        new_cond = np.concatenate([label_part, [i/2.0, j/2.0]])
        new_cond = new_cond.reshape(1, 12).astype('float32')
        new_cond_tensor = tf.convert_to_tensor(new_cond)
        out = model_best.decode([z, new_cond_tensor], apply_sigmoid=True)
        plt.imshow(out[0, :, :, 0], cmap='gray')
        plt.axis('off')
plt.show()

```

Komentarz wyników:

Model poprawnie generuje tę samą cyfrę w różnych miejscach na siatce 3x3. Obrazy pokazują, że: - Cyfra pozostaje taka sama (zgodna z etykietą) - Pozycja zmienia się zgodnie z podanymi współrzędnymi (pos_x, pos_y) - Jakość generowanych obrazów jest dobra

Wnioski: Model poprawnie rozdziela informację o cyfrze (zawartą w reprezentacji z) od informacji o pozycji (zawartej w wektorze warunkującym). To pokazuje, że model nauczył się efektywnie wykorzystywać oba wejścia.

Zadanie 3.3 - Testowanie z losową reprezentacją z (0.25pkt)

Kod:

```

label_hot = np.zeros(10); label_hot[5]=1 # Cyfra 5
z_rand = tf.random.normal((1, model_best.latent_dim))

plt.figure(figsize=(9, 9))
for i in range(3):
    for j in range(3):
        ax = plt.subplot(3, 3, 3*i+j+1)
        new_cond = np.concatenate([label_hot, [i/2.0, j/2.0]])
        new_cond = new_cond.reshape(1, 12).astype('float32')
        new_cond_tensor = tf.convert_to_tensor(new_cond)
        out = model_best.decode([z_rand, new_cond_tensor], apply_sigmoid=True)
        plt.imshow(out[0, :, :, 0], cmap='gray')
        plt.axis('off')
plt.show()

```

Komentarz wyników:

Model generuje cyfrę 5 w różnych pozycjach na siatce. Obrazy pokazują: - Wszystkie wygenerowane

obrazy przedstawiają cyfrę 5 (zgodnie z etykietą) - Pozycja cyfry zmienia się zgodnie z podanymi współrzędnymi - Jakość jest dobra, choć nieco gorsza niż przy użyciu rzeczywistej reprezentacji z

Wnioski: Model poprawnie wykorzystuje wektor warunkujący do kontrolowania zarówno typu cyfry, jak i jej pozycji. Losowa reprezentacja z działa dobrze, co pokazuje, że model nauczył się efektywnie wykorzystywać przestrzeń ukrytą.

4. Warunkowa sieć GAN (Conditioned GAN)

Zadanie 4.1 - Generowanie obrazów dla każdej cyfry (1 pkt)

Kod:

```
plt.figure(figsize=(10, 2))
for i in range(10):
    noise = tf.random.normal([1, noise_dim])
    cond = np.zeros((1, 10), dtype='float32')
    cond[0, i] = 1
    cond = tf.convert_to_tensor(cond)
    img = generator([noise, cond], training=False)
    plt.subplot(1, 10, i+1)
    plt.imshow(img[0, :, :, 0], cmap='gray')
    plt.axis('off')
    plt.title(str(i))
plt.show()
```

Parametry treningu: - Liczba epok: 30 (zoptymalizowane z 100) - Batch size: 256 - Learning rate: 1e-4 - Latent dimension: 100 - Label smoothing: 0.9 dla prawdziwych obrazów

Ocena jakości wyników:

Generator wytrenowany przez 30 epok generuje rozpoznawalne cyfry dla wszystkich klas (0-9). Obrazy są: - **Czytelne:** Większość cyfr jest łatwo rozpoznawalna - **Różnorodne:** Różne próbki szumu dają różne warianty tej samej cyfry - **Zgodne z etykietą:** Generowane cyfry odpowiadają podanym etykietom

Jakość w porównaniu do innych modeli: - **Lepsza niż AE/VAE:** GAN generuje bardziej realistyczne i ostre obrazy - **Różnorodność:** GAN generuje większą różnorodność niż VAE - **Stabilność:** Trening był stabilny dzięki label smoothing

Wnioski: Model GAN osiągnął dobrą jakość generacji. 30 epok okazało się wystarczające dla zadania na MNIST, przy zachowaniu rozsądnego czasu treningu (~30-35 minut).

Podsumowanie i wnioski

Porównanie modeli:

1. Autoenkoder (AE):

- Najprostszy model
- Ograniczone możliwości generacji nowych przykładów

- Przestrzeń ukryta nie jest ciągła
 - Najszybszy trening
2. **VAE:**
- Lepsza jakość generacji niż AE
 - Ciągła przestrzeń ukryta dzięki regularizacji
 - Możliwość interpolacji między przykładami
 - Nieco wolniejszy trening niż AE
3. **Conditioned VAE:**
- Kontrola nad generowanymi obrazami (cyfra + pozycja)
 - Dobra jakość przy odpowiedniej przestrzeni ukrytej
 - Wymaga więcej czasu treningu
4. **Conditioned GAN:**
- Najlepsza jakość generowanych obrazów
 - Największa różnorodność
 - Najdłuższy czas treningu
 - Wymaga starannego balansowania generatora i dyskryminatora

Wnioski końcowe:

- **Przestrzeń ukryta:** Większa przestrzeń ukryta (latent_dim) poprawia jakość, ale zwiększa czas treningu
- **Regularizacja:** KL-divergence w VAE zapewnia bardziej uporządkowaną przestrzeń ukrytą
- **Warunkowanie:** Dodanie informacji warunkującej pozwala na precyzyjną kontrolę generacji
- **Czas treningu:** Zoptymalizowane parametry pozwalają na trening wszystkich modeli w czasie < 1 godziny

Wszystkie modele zostały pomyślnie wytrenowane i generują sensowne wyniki zgodne z oczekiwaniami.